



DAY-11

Python Function

Python Function

- **Functions are the most important aspect of an application.** A function can be defined as the organized block of reusable code, which can be called whenever required.
- **Python allows us to divide a large program into the basic building blocks known as a function. The function contains the set of programming statements enclosed by {}.** A function can be called multiple times to provide reusability and modularity to the Python program.
- The Function helps to programmer to break the program into the smaller part. It organizes the code very effectively and avoids the repetition of the code. As the program grows, function makes the program more organized.

- Python provide us various inbuilt functions like range() or print(). Although, the user can create its functions, which can be called user-defined functions.
- **There are mainly two types of functions.**
- **User-define functions** - The user-defined functions are those define by the user to perform the specific task.
- **Built-in functions** - The built-in functions are those functions that are pre-defined in Python.
- In this tutorial, we will discuss the user define functions.

Advantage of Functions in Python

- There are the following advantages of Python functions.
- Using functions, we can avoid rewriting the same logic/code again and again in a program.
- We can call Python functions multiple times in a program and anywhere in a program.
- We can track a large Python program easily when it is divided into multiple functions.
- **Reusability is the** main achievement of Python functions.
- However, Function calling is always overhead in a Python program.

Creating a Function

- Python provides the **def** keyword to define the function. The syntax of the define function is given below.
- **Syntax:**
- **def my_function(parameters):**
- function_block
- **return** expression
- Let's understand the syntax of functions definition.
- **The def keyword, along with the function name is used to define the function.**
- **The identifier rule must follow the function name.**
- A function accepts the parameter **(argument), and they can be optional.**
- **The function block is started with the colon (:),** and block statements must be at the same indentation.
- The **return** statement is used to return the value. A function can have only one **return**

Function Calling

- In Python, after the function is created, we can call it from another function. A function must be defined before the function call; otherwise, the Python interpreter gives an error. To call the function, use the function name followed by the parentheses.
- Consider the following example of a simple example that prints the message "Hello World".
 - #function definition
 - `def hello_world():`
 - `print("hello world")`
 - # function calling
 - `hello_world()`
 - Output:
 - hello world

The return statement

- The return statement is used at the end of the function and returns the result of the function. It terminates the function execution and transfers the result where the function is called. The return statement cannot be used outside of the function.
- Syntax
- `return [expression_list]`
- It can contain the expression which gets evaluated and value is returned to the caller function. If the return statement has no expression or does not exist itself in the function then it returns the None object.

Consider the following example:

- Example 1
- # Defining function
- `def sum():`
 - `a = 10`
 - `b = 20`
 - `c = a+b`
 - `return c`
- # calling sum() function in print statement
- `print("The sum is:",sum())`
- Output:
- The sum is: 30

- In the above code, we have defined the function named sum, and it has a statement `c = a+b`, which computes the given values, and the result is returned by the return statement to the caller function.
- **Example 2 Creating function without return statement**
- **# Defining function**
- `def sum():`
 - `a = 10`
 - `b = 20`
 - `c = a+b`
- **# calling sum() function in print statement**
- `print(sum())`
- **Output:**
- `None`
- In the above code, we have defined the same function without the return statement as we can see that the sum() function returned the None object to the caller function.

Arguments in function

- The arguments are types of information which can be passed into the function. The arguments are specified in the parentheses. We can pass any number of arguments, but they must be separate them with a comma.
- Consider the following example, which contains a function that accepts a string as the argument.
- Example 1
- #defining the function
- `def func (fname,lname):`
- `print("Hi ",fname,lname)`
- #calling the function
- `func(fname:"Devansh")`
- Output:
- Hi Devansh

Example 2

- #Python function to calculate the sum of two variables
- #defining the function
- `def sub (a,b):`
- `return a-b;`
-
- #taking values from the user
- `a = int(input("Enter a: "))`
- `b = int(input("Enter b: "))`
-
- #printing the sum of a and b
- `print("sub = ",sub(a,b))`
- Output:
- Enter a: 10
- Enter b: 20
- Sub = -10

Call by reference in Python

- In Python, call by reference means passing the actual value as an argument in the function. All the functions are called by reference, i.e., all the changes made to the reference inside the function revert back to the original value referred by the reference.
- Example 1 Passing Immutable Object (List)
- #defining the function
- `def change_list(list1):`
- `list1.append(20)`
- `list1.append(30)`
- `print("list inside function = ",list1)`
-
- #defining the list
- `list1 = [10,30,40,50]`
-
- #calling the function
- `change_list(list1)`
- `print("list outside function = ",list1)`

- Output:
- list inside function = [10, 30, 40, 50, 20, 30]
- list outside function = [10, 30, 40, 50, 20, 30]

Example 2 Passing Mutable Object (String)

- #defining the function
- def change_string (str):
- str = str + " Hows you "
- print("printing the string inside function :",str)
-
- string1 = "Hi I am there"
-
- #calling the function
- change_string(string1)
-
- print("printing the string outside function :",string1)
- Output:
-
- printing the string inside function : Hi I am there Hows you
- printing the string outside function : Hi I am there

Types of arguments

- There may be several types of arguments which can be passed at the time of function call.
- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Required Arguments

- Till now, we have learned about function calling in Python. However, we can provide the arguments at the time of the function call. **As far as the required arguments are concerned, these are the arguments which are required to be passed at the time of function calling with the exact match of their positions in the function call and function definition.** If either of the arguments is not provided in the function call, or the position of the arguments is changed, the Python interpreter will show the error.
- Consider the following example.

- Example 1
- `def func(name):`
- `message = "Hi "+name`
- `return message`
- `name = input("Enter the name:")`
- `print(func(name))`
- Output:
- Enter the name: John
- Hi John

Example 2

- #the function simple_interest accepts three arguments and returns the simple interest accordingly
- def simple_interest(p,t,r):
- return (p*t*r)/100
- p = float(input("Enter the principle amount? "))
- r = float(input("Enter the rate of interest? "))
- t = float(input("Enter the time in years? "))
- print("Simple Interest: ",simple_interest(p,r,t))

- Output:
- Enter the principle amount: 5000
- Enter the rate of interest: 5
- Enter the time in years: 3
- Simple Interest: 750.0

Example 3

- #the function calculate returns the sum of two arguments a and b
- `def calculate(a,b):`
- `return a+b`
- `calculate(10)` # this causes an error as we are missing a required arguments b.
- Output:
- `TypeError: calculate() missing 1 required positional argument: 'b'`

Default Arguments

- Python allows us to initialize the arguments at the function definition. If the value of any of the arguments is not provided at the time of function call, then that argument can be initialized with the value given in the definition even if the argument is not specified at the function call.

- Example 1
- `def printme(name,age=22):`
- `print("My name is",name,"and age is",age)`
- `printme(name = "john")`
- Output:
- My name is John and age is 22

Example 2

- `def printme(name,age=22):`
- `print("My name is",name,"and age is",age)`
- `printme(name = "john")` #the variable age is not passed into the function however the default value of age is considered in the function
- `printme(age = 10,name="David")` #the value of age is overwritten here, 10 will be printed as age
- Output:
- My name is john and age is 22
- My name is David and age is 10

Variable-length Arguments (*args)

- In large projects, sometimes we may not know the number of arguments to be passed in advance. In such cases, Python provides us the flexibility to offer the comma-separated values which are internally treated as tuples at the function call. By using the variable-length arguments, we can pass any number of arguments.
- However, at the function definition, we define the variable-length argument using the *args (star) as * <variable - name >.
- Consider the following example.

Example

- `def printme(*names):`
- `print("type of passed argument is ",type(names))`
- `print("printing the passed arguments...")`
- `for name in names:`
- `print(name)`
- `printme("john","David","smith","nick")`

- Output:

- type of passed argument is <class 'tuple'>
- printing the passed arguments...
- john
- David
- smith
- nick

- In the above code, we passed *names as variable-length argument. We called the function and passed values which are treated as tuple internally. The tuple is an iterable sequence the same as the list. To print the given values, we iterated *arg names using for loop.

Keyword arguments(**kwargs)

- Python allows us to call the function with the keyword arguments. This kind of function call will enable us to pass the arguments in the random order.
- The name of the arguments is treated as the keywords and matched in the function calling and definition. If the same match is found, the values of the arguments are copied in the function definition.
- Consider the following example.

Example 1

- #function func is called with the name and message as the keyword arguments
- `def func(name,message):`
 - `print("printing the message with",name,"and ",message)`
 -
- #name and message is copied with the values John and hello respectively
- `func(name = "John",message="hello")`
- Output:
 - printing the message with John and hello

- Example 2 providing the values in different order at the calling
- #The function `simple_interest(p, t, r)` is called with the keyword arguments the order of arguments doesn't matter in this case
- **`def simple_interest(p,t,r):`**
- **`return (p*t*r)/100`**
- **`print("Simple Interest: ",simple_interest(t=10,r=10,p=1900))`**
- Output:
- Simple Interest: 1900.0
- If we provide the different name of arguments at the time of function call, an error will be thrown.

Consider the following example.

- Example 3
- #The function `simple_interest(p, t, r)` is called with the keyword arguments.
- `def simple_interest(p,t,r):`
- `return (p*t*r)/100`
-
- # **doesn't find the exact match of the name of the arguments (keywords)**
- `print("Simple Interest: ",simple_interest(time=10,rate=10,principle=1900))`
- Output:
- `TypeError: simple_interest() got an unexpected keyword argument 'time'`

- The Python allows us to provide the mix of the required arguments and keyword arguments at the time of function call. However, the required argument must not be given after the keyword argument, i.e., once the keyword argument is encountered in the function call, the following arguments must also be the keyword arguments.

Consider the following example.

- Example 4
- `def func(name1,message,name2):`
- `print("printing the message with",name1,",",message,",and",name2)`
- `#the first argument is not the keyword argument`
- `func("John",message="hello",name2="David")`
- Output:
- printing the message with John , hello ,and David
- The following example will cause an error due to an in-proper mix of keyword and required arguments being passed in the function call.

- Example 5
- `def func(name1,message,name2):`
- `print("printing the message with",name1,",",message,",and",name2)`
- `func("John",message="hello","David")`
- Output:
- `SyntaxError: positional argument follows keyword argument`

- Python provides the facility to pass the multiple keyword arguments which can be represented as `**kwargs`. It is similar as the `*args` but it stores the argument in the dictionary format.
- This type of arguments is useful when we do not know the number of arguments in advance.
- Consider the following example:

Example 6: Many arguments using Keyword argument

- `def food(**kwargs):`
- `print(kwargs)`
- `food(a="Apple")`
- `food(fruits="Orange", Vagitable="Carrot")`
- Output:
- `{'a': 'Apple'}`
- `{'fruits': 'Orange', 'Vagitable': 'Carrot'}`

Scope of variables

- The scopes of the variables depend upon the location where the variable is being declared. The variable declared in one part of the program may not be accessible to the other parts.
- In python, the variables are defined with the two types of scopes.
 - Global variables
 - Local variables
- The variable defined outside any function is known to have a global scope, whereas the variable defined inside a function is known to have a local scope.

Consider the following example.

- Example 1 Local Variable
- `def print_message():`
- `message = "hello !! I am going to print a message." # the variable message is local to the function itself`
- `print(message)`
- `print_message()`
- `print(message) # this will cause an error since a local variable cannot be accessible here.`
- Output:
- `hello !! I am going to print a message.`
- `File "/root/PycharmProjects/PythonTest/Test1.py", line 5, in`
- `print(message)`
- `NameError: name 'message' is not defined`

Example 2 Global Variable

- `def calculate(*args):`
- `sum=0`
- `for arg in args:`
- `sum = sum +arg`
- `print("The sum is",sum)`
- `sum=0`
- `calculate(10,20,30)` #60 will be printed as the sum
- `print("Value of sum outside the function:",sum)` # 0 will be printed Output:
- Output:
- The sum is 60
- Value of sum outside the function: 0

Python Built-in Functions

- The Python built-in functions are defined as the functions whose functionality is pre-defined in Python. The python interpreter has several functions that are always present for use. These functions are known as Built-in Functions. There are several built-in functions in Python which are listed below:

Python abs() Function

- The python abs() function is used to return the absolute value of a number. It takes only one argument, a number whose absolute value is to be returned. The argument can be an integer and floating-point number. If the argument is a complex number, then, abs() returns its magnitude.
- Python abs() Function Example

- # integer number
- integer = -20
- print('Absolute value of -40 is:', abs(integer))
-
- # floating number
- floating = -20.83
- print('Absolute value of -40.83 is:', abs(floating))
- Output:
- Absolute value of -20 is: 20
- Absolute value of -20.83 is: 20.83

- Python all() Function

- The python all() function accepts an iterable object (such as list, dictionary, etc.). It returns true if all items in passed iterable are true. Otherwise, it returns False. If the iterable object is empty, the all() function returns True.

- Python all() Function Example

- # all values true

- k = [1, 3, 4, 6]

- print(all(k))

-

- # all values false

- k = [0, False]

- print(all(k))

-

- # one false value

- k = [1, 3, 7, 0]

- print(all(k))

-

- # one true value

- k = [0, False, 5]

- print(all(k))

-

- # empty iterable

- k = []

- print(all(k))

- Output:

- True
- False
- False
- False
- True

- Python bin() Function
- The python bin() function is used to return the binary representation of a specified integer. A result always starts with the prefix 0b.
- Python bin() Function Example
- `x = 10`
- `y = bin(x)`
- `print (y)`
- Output:
- 0b1010

- Python bool()
- The python bool() converts a value to boolean(True or False) using the standard truth testing procedure.

- Python bool() Example

- test1 = []
- print(test1,'is',bool(test1))
- test1 = [0]
- print(test1,'is',bool(test1))
- test1 = 0.0
- print(test1,'is',bool(test1))
- test1 = None
- print(test1,'is',bool(test1))
- test1 = True
- print(test1,'is',bool(test1))
- test1 = 'Easy string'
- print(test1,'is',bool(test1))

- Output:

- [] is False
- [0] is True
- 0.0 is False
- None is False
- True is True
- Easy string is True

- Python bytes()
- The python bytes() in Python is used for returning a bytes object. It is an immutable version of the bytearray() function.
- It can create empty bytes object of the specified size.
- Python bytes() Example
 - string = "Hello World."
 - array = bytes(string, 'utf-8')
 - print(array)
 - Output:
 - b ' Hello World.'

Python callable() Function

- A python callable() function in Python is something that can be called. This built-in function checks and returns true if the object passed appears to be callable, otherwise false.
- Python callable() Function Example
 - `x = 8`
 - `print(callable(x))`
 - Output:
 - False

Python exec() Function

- The python `exec()` function is used for the dynamic execution of Python program which can either be a string or object code and it accepts large blocks of code, unlike the `eval()` function which only accepts a single expression.
- Python `exec()` Function Example
 - `x = 8`
 - `exec('print(x==8)')`
 - `exec('print(x+4)')`
 - Output:
 - True
 - 12

Python sum() Function

- As the name says, python sum() function is used to get the sum of numbers of an iterable, i.e., list.
- Python sum() Function Example
 - `s = sum([1, 2,4 ])`
 - `print(s)`
 -
 - `s = sum([1, 2, 4], 10)`
 - `print(s)`
 - Output:
 - 7
 - 17

- Python any() Function
- The python any() function returns true if any item in an iterable is true. Otherwise, it returns False.
- Python any() Function Example
- l = [4, 3, 2, 0]
- print(any(l))
-
- l = [0, False]
- print(any(l))
-
- l = [0, False, 5]
- print(any(l))
-
- l = []
- print(any(l))

- Output:

- True

- False

- True

- False

Python ascii() Function

- The python `ascii()` function returns a string containing a printable representation of an object and escapes the non-ASCII characters in the string using `\x`, `\u` or `\U` escapes.
- Python `ascii()` Function Example
- `normalText = 'Python is interesting'`
- `print(ascii(normalText))`
-
- `otherText = 'Pythön is interesting'`
- `print(ascii(otherText))`
-
- `print('Pyth\x66n is interesting')`

- Output:
- 'Python is interesting'
- 'Pyth\x66n is interesting'
- Python is interesting

Python bytearray()

- The python bytearray() returns a bytearray object and can convert objects into bytearray objects, or create an empty bytearray object of the specified size.
- Python bytearray() Example
- string = "Python is a programming language."
-
- # string with encoding 'utf-8'
- arr = bytearray(string, 'utf-8')
- print(arr)
- Output:
- bytearray(b'Python is a programming language.')

Python eval() Function

- The python eval() function parses the expression passed to it and runs python expression(code) within the program.
- Python eval() Function Example
 - `x = 8`
 - `print(eval('x + 1'))`
 - Output:
 - 9

Python format() Function

- The python format() function returns a formatted representation of the given value.
- Python format() Function Example
 - # d, f and b are a type
 -
 - # integer
 - `print(format(123, "d"))`
 -
 - # float arguments
 - `print(format(123.4567898, "f"))`
 -
- # binary format
- `print(format(12, "b"))`
- Output:
 - 123
 - 123.456790
 - 1100

Day Tasks

- **Exercise 1: Create a function in Python**
- Write a program to create a function that takes two arguments, name and age, and print their value.
- Hint:
- Use the def keyword with the function name to define a function.
- Next, take two parameters
- Print them using the print() function
- Call function by passing name and age

Exercise 2:

- Create a function with variable length of arguments
- Write a program to create function func1() to accept a variable length of arguments and print their value.
- Note: Create a function in such a way that we can pass any number of arguments to this function, and the function should process them and display each argument's value.

Exercise 3

- : Return multiple values from a function
- Write a program to create function calculation() such that it can accept two variables and calculate addition and subtraction. Also, it must return both addition and subtraction in a single return call.

Exercise 4:

- Create a function with a default argument
- Write a program to create a function `show_employee()` using the following conditions.
- It should accept the employee's name and salary and display both.
- If the salary is missing in the function call then assign default value 9000 to salary

This PPT contents are

TO

© copyright
all rights reserved

Domain®
Computer Education
TRAINING & PLACEMENT CENTRE



TRAINING & PLACEMENT CENTRE

DOMAIN COMPUTER EDUCATION

2nd floor, Balaji Celebration, Ganpati
Mandir Road, Vishrambagh, Sangli

7620575756



DOMAIN COMPUTER EDUCATION

3rd Floor Prabhakar Plaza, Station road
Kolhapur.

Ph.No.- 9225804090



Website: www.domaincomputer.org